

DataMaestro: A Versatile and Efficient Data Streaming Engine Bringing Decoupled Memory Access To Dataflow Accelerators

Xiaoling Yi^{1*}, Yunhao Deng^{1*}, Ryan Antonio¹, Fanchen Kong¹, Guilherme Paim², Marian Verhelst¹

¹MICAS-ESAT, KU Leuven, Leuven, Belgium

²INESC-ID, Instituto Superior Técnico, Universidade de Lisboa, Lisboa, Portugal

Abstract—Deep Neural Networks (DNNs) have achieved remarkable success across various intelligent tasks but encounter performance and energy challenges in inference execution due to data movement bottlenecks. We introduce *DataMaestro*, a versatile and efficient data streaming unit that brings the decoupled access/execute architecture to DNN dataflow accelerators to address this issue. *DataMaestro* supports flexible and programmable access patterns to accommodate diverse workload types and dataflows, incorporates fine-grained prefetch and addressing mode switching to mitigate bank conflicts, and enables customizable on-the-fly data manipulation to reduce memory footprints and access counts. We integrate five *DataMaestros* with a Tensor Core-like GeMM accelerator and a Quantization accelerator into a RISC-V host system for evaluation. The FPGA prototype and VLSI synthesis results demonstrate that *DataMaestro* helps the GeMM core achieve nearly 100% utilization, which is 1.05–21.39× better than state-of-the-art solutions, while minimizing area and energy consumption to merely 6.43% and 15.06% of the total system.

I. INTRODUCTION

Deep Neural Networks (DNNs) have significantly gained popularity for tackling complex intelligence tasks and potentially advancing humanity toward the Artificial General Intelligence (AGI) era. To meet the stringent performance and energy consumption requirements of DNN deployment, numerous contemporary domain-specific dataflow accelerators have been developed [2], [7]–[11], incorporating extensive optimizations in the processing element (PE) array design and dataflow strategies. However, as DNN workloads and PE arrays continue to scale, data communication between the accelerator datapath and the memory hierarchy emerges as a critical bottleneck in DNN dataflow accelerators [4], [12]–[16].

Despite various solutions having been proposed to address this bottleneck, three shortcomings hinder their widespread deployment toward efficient DNN dataflow accelerators: **1). Non-Reusable Design.** Data movement units are typically implemented as dedicated hardware blocks, tightly coupled to a specific accelerator, featuring fixed access patterns and bandwidths tuned to specific workloads [2]–[4], [6], [7]. Given the diverse sizes, topologies, and rapid evolution of DNN workloads, these dedicated data movement units lack the versatility to adapt to different dataflows' requirements, leading to suboptimal overall system performance and energy efficiency. For instance, BitWave [2] achieves very high PE array utilization for convolution operations in convolutional neural networks (CNNs) by leveraging specialized optimizations but falls short in general matrix-matrix multiplication (GeMM), which is pervasive in Transformer models. Moreover, the inability to reuse these dedicated data movement blocks across different accelerators results in redundant engineering efforts and hinders competitive time-to-market, especially considering the rapid evolution of DNNs and next-generation computing platforms. **2). Performance Decrease Caused by Bank Conflicts.** The on-chip memory bank conflict challenge is a primary obstacle to achieving peak system performance. For instance, Gemmini [1] exhibits a PE array utilization as low as 10% due to the lack of bank conflict management in the access of the different operands. To address this problem, many accelerators [2], [8], [17], [18] leverage private on-chip memory for each operand, such as dedicated input, weight, and output buffers.

However, bank conflicts still arise when data needed in a single cycle is stored in different wordlines within the same bank, a scenario commonly encountered in strided convolution operations. Moreover, it imposes additional limitations on the tiling of the workload to meet the requirement of the smallest memory and introduces extra complexity to manage multiple address spaces [15]. **3). Expensive Data Manipulation.** Many accelerators require preprocessing of operand before computation, including permutation, data alignment, and *im2col*. A common approach is to design standalone data manipulation units to perform these data rearrangement operations [1], [9], [11]. These units require additional memory accesses and intermediate data storage, which again increases the likelihood of memory conflicts and elevates memory energy consumption. In conclusion, there is an absence of a versatile and efficient data movement solution to address the diverse dataflow and data manipulation requirements of different accelerators while also ensuring high system performance.

The decoupled access/execute (DAE) architecture [19] separates data access and computation processes into two independent streams to prevent mutual stalls. This approach has been applied in numerous works [5], [20], [21], demonstrating performance speed-ups of 2–5× for linear algebra workloads [5], [20]. The DAE architecture is well-suited for DNN workloads, as its regular multidimensional access patterns [22] enable data to be fetched ahead of computation. This facilitates continuous data stream consumption and production from the perspective of PE arrays, helping the accelerator system reach its theoretical peak performance. Previous works [6], [7] have successfully applied the DAE architecture in accelerator designs. However, they fail to meet the diverse dataflow, data access patterns, and high bandwidth requirements of DNN workloads and the aforementioned challenges, rendering them suboptimal for DNN dataflow accelerators.

In this paper, we present a versatile and efficient data streaming engine, called *DataMaestro*¹, which brings decoupled memory access to DNN dataflow accelerators. *DataMaestro* is highly optimized and can be reused across different dataflow accelerators with flexible and programmable access patterns while achieving nearly 100% PE array utilization. A comprehensive comparison between *DataMaestro* and state-of-the-art (SotA) solutions is listed in Table I. In summary, our main contributions lie in:

- We present *DataMaestro*, a generalized, versatile, and efficient data streaming engine for diverse DNN dataflow accelerators, with design-time and runtime configurability, enabling decoupled memory access for arbitrary dataflows (§ III-A and § III-B).
- We introduce fine-grained prefetch and runtime-configurable addressing mode switching features inside *DataMaestro*, which minimizes bank conflicts and fully exploits the available on-chip memory bandwidth (§ III-C and § III-D).
- We provide on-the-fly data manipulation capability inside the *DataMaestro* by customizable datapath extensions to effectively reduce memory footprints and access counts (§ III-E).

¹*DataMaestro* and the *DataMaestro*-boosted accelerator system are open-sourced at: https://github.com/KULeuven-MICAS/snax_cluster.

*Equal contribution.

TABLE I: Comparison of the SotA data movement solutions with *DataMaestro*.

	Gemmini [1]	BitWave [2]	[3]	FEATHER [4]	SSR [5]	Buffet [6]	Softbrain [7]	<i>DataMaestro</i>
Open source	✓	✗	✗	✓	✓	✓	✗	✓
Reusable design	✗	✗	✗	✗	✗	✓	✗	✓
Decoupled access/execute	✗	✗	✗	✗	✓	✓	✓	✓
Programmable affine access	✓(2-D)	✗	✓(2-D)	✗	✓(4-D)	✓(2-D)	✓(2-D)	✓(N-D)
Fine-grained prefetch	✗	✗	✗	✗	✗	✗	✗	✓
Runtime addressing mode switching	✗	✗	✗	✗	✗	✗	✗	✓
On-the-fly data manipulation	✗	✗	✗	✓	✗	✗	✗	✓

- We demonstrate the versatility and efficiency of *DataMaestro* by integrating it with a Tensor core [23]-like GeMM accelerator and a Quantization accelerator into a RISC-V host system. The FPGA prototype and VLSI synthesis results show that *DataMaestro* enhances the GeMM PE array utilization to 95.45%-99.98% under real-world DNN workloads, boosting normalized throughput to 1.05-21.39× over SotA, while only consuming 6.43% and 15.06% of system area and energy (§ IV).

II. BACKGROUND

A. Dataflow and Data Layout

Typical DNN kernels, such as GeMM, multi-head attention, convolution, and pooling, can be represented as nested loops that iterate over the input, weight, and output tensor dimensions. Dataflow optimizations, like splitting and reordering these loops, open up a vast mapping space for efficient hardware processing [14], [15]. Specifically, these optimizations include spatial unrolling (SU) for parallel execution and temporal unrolling (TU) to control data tiling and the order of loop iterations.

The data access pattern, which specifies the data required by the accelerator in each clock cycle, is inherently determined by the dataflow. However, the data layout, referring to the organization of tensor data in memory, also influences data access patterns. A proper data layout is crucial for optimizing DNN performance, as it directly impacts memory access efficiency and can lead to PE array underutilization if mismatched with the dataflow [4], [24].

B. Decoupled Access/Execute Architecture for Dataflow Accelerators

A decoupled access/execute (DAE) architecture [6], [7], [19] explicitly decouples memory access and computation by introducing data streaming engines between them, as illustrated in Figure 1. This architecture integrates three key components — memory subsystem, data streaming engine, and accelerator datapath—which collaborate coherently to produce/consume three types of streams: access streams, data streams, and execute streams. The memory subsystem serves as a data reservoir, responding to the memory requests via the access stream. The accelerator’s datapath applies computation to the data streams and produces results via the execute stream. The data streaming engine bridges the memory subsystem with the accelerator datapath, orchestrating data flow between them. It transforms scattered data in memory which is arranged in a specific format (data layout), into a continuous data stream, which is organized in the format required by the accelerator (data flow) and vice versa, by interacting with the memory through the access stream.

Typically, the streaming engine consists of three main components: an Address Generation Unit (AGU), a memory interface controller, and a data FIFO. The AGU generates address sequences, which are then consumed by the memory interface controller to perform memory operations. The data FIFO temporarily stores data streams, decoupling memory access from computation. This decoupling enables data prefetch operations and thus minimizes computation interruptions caused by memory stalls. Such an approach is particularly beneficial

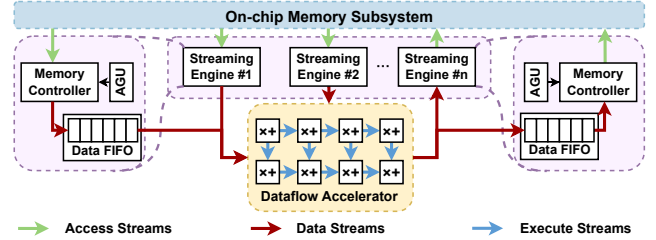


Fig. 1: Overview of decoupled access/execute architecture.

for DNN workloads due to their regular and deterministic data access patterns. By incorporating this architecture, a harmonious access-data-execute stream orchestration is established within the accelerator system. In the next section, we present *DataMaestro*, a highly optimized data streaming engine designed for DNN accelerators, which efficiently streamlines the data streams to enhance the performance of dataflow accelerators.

III. *DataMaestro* ARCHITECTURE

In this section, we will first provide an overview of the *DataMaestro* architecture in § III-A, along with all of its configurable parameters, which are listed in Table II. Next, we introduce the three novel architectural features that maximize *DataMaestro*’s efficiency and minimize memory bank conflicts across a wide range of workloads: 1) highly configurable and area-efficient N-Dimensional (N -D) address generation unit (§ III-B); 2) fine-grained prefetch (§ III-C); and 3) runtime-configurable addressing mode switching (§ III-D). Finally, we discuss the datapath extension interface of *DataMaestro*, which enables customized on-the-fly data manipulation (§ III-E). These unique features of *DataMaestro* are key to its high efficiency and enhancement of the performance of DNN dataflow accelerators.

 TABLE II: Design-time parameters and runtime configurations in *DataMaestro*.

Scope	Parameter	Description	Type [†]
Design Time	N_R	Num. of read <i>DataMaestro</i>	Int
	N_W	Num. of write <i>DataMaestro</i>	Int
	$Mode_{R/W}$	Read or Write mode	Bool.
	B_s	Spatial Bounds	[Int]
	D_s	Num. of Spatial Dimensions	Int
	D_t	Num. of Temporal Dimensions	Int
	N_C	Num. of Channels	Int
	D_{ABf}	Address Buffer Depth	Int
	D_{DBf}	Data Buffer Depth	Int
	DP_{ext}	Datapath Extensions	[Ext.]
	W_B	Bank Width	Int
	N_{BF}	Num. of Bank in Full Memory	Int
	N_{BG}	Num. of Bank in One Group	[Int]
Run Time	$Addr_B$	Base Address	UInt
	S_s	Spatial Strides	[UInt]
	B_t	Temporal Bounds	[UInt]
	S_t	Temporal Strides	[UInt]
	R_s	Addressing Mode Selection	UInt

[†] Type enclosed in square brackets [] means List.

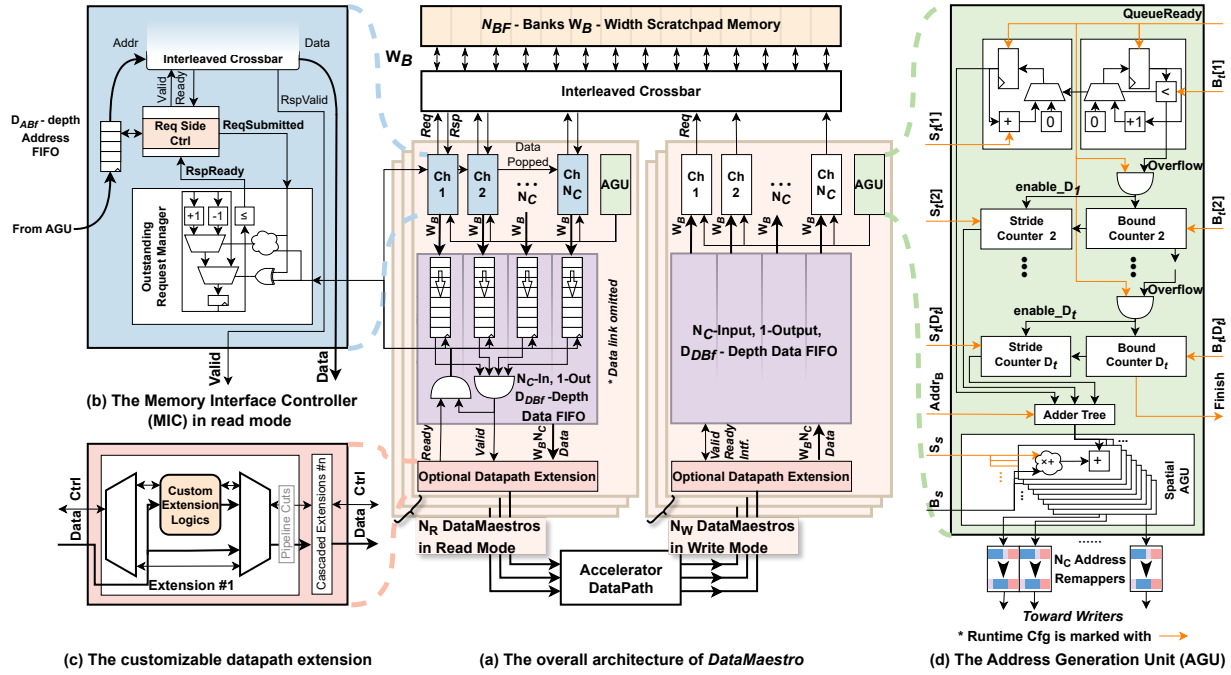


Fig. 2: DataMaestro architecture.

A. Architecture Overview of DataMaestro

Figure 2 (a) illustrates a scenario where $N_R + N_W$ DataMaestros are connected to a multi-banked scratchpad memory subsystem (top) and an accelerator (bottom), effectively managing data streams between them to create a complete accelerator system. One read DataMaestro (middle, left) and one write DataMaestro (middle, right) are depicted in detail. The memory subsystem consists of an N_B -banked scratchpad memory that provides high memory bandwidth, along with an interleaved crossbar that ensures full accessibility from any request port. To meet the specific requirements of different accelerator ports, N_R read and N_W write DataMaestros are deployed, each of which can be configured independently at both design time and runtime, by providing the parameters listed in Table II.

A read or write DataMaestro primarily includes N_C independent memory interaction channels, breaking one wide memory request into multiple narrower operations for asynchronous, fine-grained memory access. Each channel is equipped with a dedicated Memory Interface Controller (MIC) (Figure 2 (b)) and a data FIFO. The MIC consumes addresses from the address generation unit (AGU) (Figure 2 (d)) to issue memory requests, and the data FIFO acts as a buffer between the DataMaestro and the accelerator. This facilitates a decoupled access/execute architecture for dataflow accelerators to effectively hide memory latency. At the output ports of the FIFOs, data fetched by multiple channels are gathered into a wide data stream and sent as a single wide data word to the accelerator datapath, enabling highly paralleled execution. Furthermore, at the interface between the DataMaestro and the accelerator (Figure 2 (c)), multiple user-customizable datapath extensions can optionally be inserted to perform on-the-fly data processing in a cascaded manner without requiring intermediate memory storage. In the following subsections, we will delve into each part of DataMaestro in more detail.

B. Programmable Affine Access Pattern by Address Generation Unit

The data access pattern for DNN workloads, determined by the dataflow and data layout, can typically be represented as an affine pattern [22]. Furthermore, DNN workloads encompass a

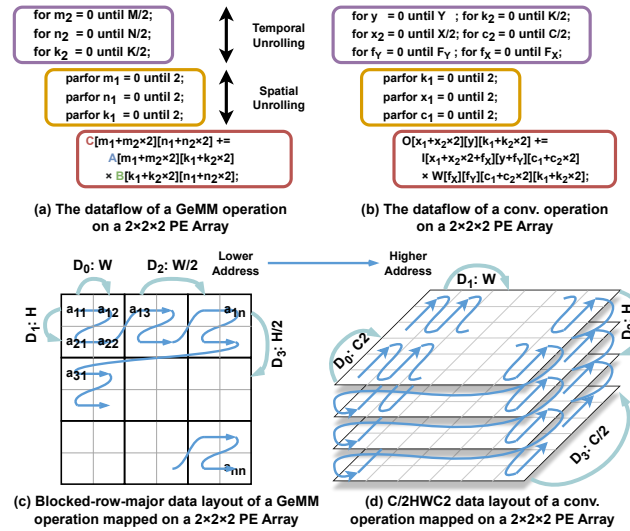


Fig. 3: Dataflow and data layout of GeMM and convolution operations mapped on $2 \times 2 \times 2$ PE array.

wide range of operation types (e.g., GeMM and various types of convolutional operations) and tensor shapes (e.g., height, width, and channel dimensions) [25], necessitating data access patterns with high flexibility. For example, as illustrated in Figure 3, the dataflow and data layout of GeMM and convolution workloads mapped on a simple $2 \times 2 \times 2$ PE array already differ significantly. For GeMM workload, the matrix A is stored using a 4-D block-row-major data layout [26], [27] (Figure 3 (c)), and exhibits a 3-D access pattern across different cycles (Figure 3 (a)). In contrast, for convolution, the input tensor is stored using a 4-D blocked data layout (Figure 3 (d)), i.e., C/2HWC2, and features a 6-D data access pattern (Figure 3 (b)). The diverse access pattern (while remaining affine) in DNN workloads motivates the design of a flexible and programmable AGU within DataMaestro.

The AGU in DataMaestro enables N -Dimensional (N -D) pro-

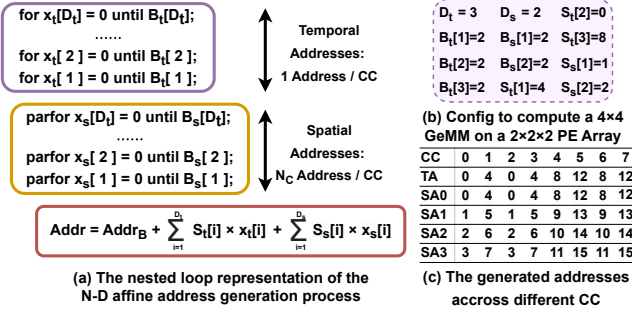


Fig. 4: The nested loop representation of the N -D affine address generation in *DataMaestro*'s AGU and a simple address generation example for a $M=N=K=4$ GeMM workload mapped on a $2 \times 2 \times 2$ PE array.

grammable affine address generation as shown in Figure 4 (a), using symbols from Table II. This affine function effectively describes the mapping of the N -D data access space to the 1-D address space. Our address generation process involves sequentially generating temporal addresses (TA) to represent temporal data access patterns and simultaneously generating multiple spatial addresses (SA) based on every temporal address for parallel data access, satisfying the requirements of temporal and spatial unrolling for a specific dataflow. A detailed example of the AGU configuration for an $M=N=K=4$ GeMM workload mapped on a $2 \times 2 \times 2$ PE array is presented in Figure 4 (b), with the corresponding address generation process depicted in Figure 4 (c).

Directly using a single counter to track the address generation progress necessitates modulo units and dividers to compute loop indices, followed by multipliers and adders to calculate temporal addresses. However, this leads to long combinatorial paths and significant hardware overheads that linearly scale with the number of dimensions. In *DataMaestro*, we incorporate microarchitectural optimizations to enable high-dimensional address generation with low overhead and shorter combinatorial paths. As shown at the top of Figure 2 (d), each dimension of the temporal AGU is implemented in a dual-counter structure, including a simple counter (Bound Counter) to record the loop index and a step-programmable counter (Stride Counter) to generate the temporal address offset. Then, address offsets of all dimensions are summed with the base address (Addr_B) and consumed by a multi-channel spatial AGU (the bottom of Figure 2 (d)), which generates distinct addresses for each channel. Based on this optimized hardware architecture, *DataMaestro* supports specifying arbitrary temporal address loop dimension, spatial address loop dimension, and spatial address loop bounds at design time, while the strides for both temporal and spatial addresses can be programmed at runtime, providing full flexibility to accommodate diverse data access patterns in DNN dataflow accelerators.

C. Fine-grained Prefetch by Memory Interface Controller

DataMaestro conducts fine-grained prefetch by breaking one wide memory request into multiple narrower channels and issuing them independently. During this process, Memory Interface Controllers (MICs) in read mode play a key role in sending fine-grained requests, gathering response data, and storing it in the data FIFO. The MIC in read mode includes an Outstanding Request Manager (ORM) and a Request Side Controller (RSC), as shown in Figure 2(b). The ORM tracks the utilization of the data FIFO and reserves the FIFO slots for in-flight memory requests, throttling the RSC if no available slots in the FIFO can be reserved. With permission from the ORM and valid

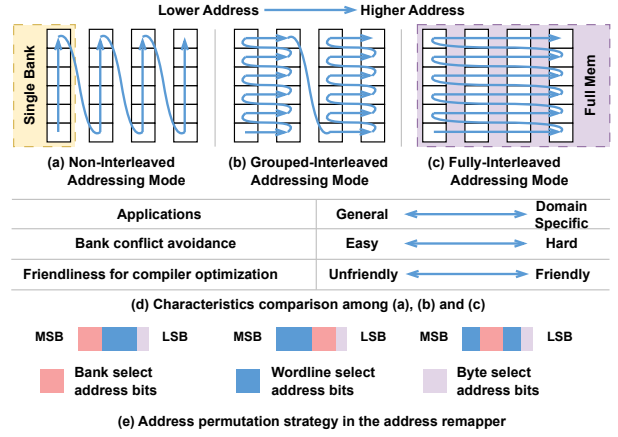


Fig. 5: Addressing modes supported by *DataMaestro*.

addresses from the AGU, the RSC immediately issues new memory requests. This aggressive and fine-grained data prefetch mechanism maximizes memory bandwidth utilization.

D. Low Overhead Addressing Mode Switching by Address Remapper

Two addressing modes are commonly used in computer systems for accessing multi-banked memory: Fully-Interleaved Memory Addressing (FIMA), where addresses are interleaved across all banks (Figure 5(a)), and Non-Interleaved Memory Addressing (NIMA), where contiguous addresses are allocated to a single bank (Figure 5(c)). Between these two lies an intermediate mode, termed Grouped-Interleaved Memory Addressing (GIMA) by us, where some banks are grouped, and the address is interleaved intra-group and remains contiguous inter-group (Figure 5(b)). The characteristics of these three addressing modes are summarized in 5 (d). While FIMA is widely used in general-purpose computing systems [28], it is more prone to bank conflicts in domain-specific accelerator (DSA) systems. As a result, contemporary dataflow accelerators often favor the NIMA mode [1]–[3]. However, the compiler needs to carefully allocate data for maximal performance and it constrains the tilings of the workload to meet the smallest memory requirement. Positioned between these two, GIMA strikes a trade-off between them. To accommodate diverse DNN workloads and dataflow accelerators, *DataMaestro* is designed to support all three addressing modes, with the capability to switch between them at runtime, offering the designer an extra degree of freedom to optimize the data allocation.

The main challenge of addressing mode switching is the overhead introduced in address decoding. However, addressing mode switching can be represented as a simple bit permutation of the address when the bank number in one group is a power of two. Based on this insight, *DataMaestro* leverages a memory address remapper that facilitates simple address bit permutation for addressing mode switching, as illustrated in Figure 5 (e). *DataMaestro* instantiates the bit permutations in hardware based on the number of banks in total (N_{BF}) and the number of banks in one group (N_{BG}) at design time. Then, the permuted addresses are connected to a multiplexer, allowing the user to select the addressing mode by setting R_5 at the run time.

E. On-the-fly Data Manipulation by Datapath Extension

DataMaestro provides a mechanism to insert customizable extensions, such as data quantization and permutation, between *DataMaestro* and accelerator datapath in a plug-and-play manner to conduct on-the-fly data manipulation, as shown in Figure 2 (c). Datapath extensions are instantiated based on the design-time configuration DP_{ext} , with the output of one extension serving as the

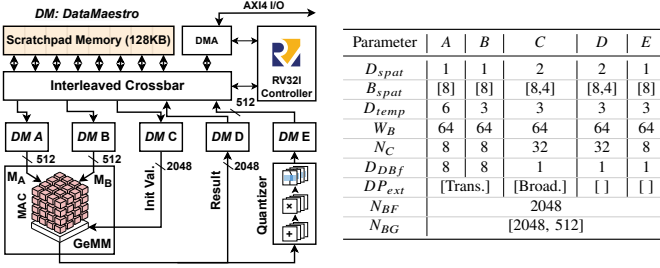


Fig. 6: The *DataMaestro* evaluation system (left) and *DataMaestros* design-time parameters used in it (right).

input for the next. Furthermore, the logic to bypass any extension is automatically inserted so that users can disable any extension at runtime.

IV. EVALUATION

A. Setup and Methodology

To assess the versatility and efficiency of *DataMaestro*, we integrate it with a Tensor Core [23]-like GeMM accelerator and a Quantization accelerator into a RISC-V host system [29], as depicted in Figure 6 (left). The GeMM accelerator features a 3-D $8 \times 8 \times 8$ PE array, capable of executing workloads expressed as $D_{32} = A_8 \otimes B_8 + C_{32}$, where the subscripts denote operand data precision and $\otimes$ signifies either GeMM or convolution operations, depending on the runtime configuration of *DataMaestros*. The output from the GeMM accelerator can be directed either to the Quantization accelerator for post-processing, represented as $E_8 = \text{Rescale}(D_{32})$, or sent back to the memory. The GeMM and Quantization accelerators feature five data stream ports in total, each with different data access patterns and bandwidth requirements, served by five *DataMaestro*, as shown in Figure 6 (right). The 6-D temporal AGU within *DataMaestro A* facilitates implicit *im2col* transformation [30] for convolution operations. Two datapath extensions are implemented: 1). Transposer, which performs on-the-fly matrix tile transposition, and 2). Broadcaster, which duplicates data across channels, particularly useful for DNN workloads with per-channel quantization. Additionally, a customized compiler is developed to generate runtime configurations for these *DataMaestros*, considering workload specifications and tensor data layouts.

We implement *DataMaestro*, GeMM, and Quantization accelerators at the register-transfer level (RTL) using Chisel [31]. An ablation study is conducted using Verilator for cycle-accurate RTL simulation to evaluate utilization gains and memory access count reductions provided by each feature of *DataMaestro* (§ IV-B). Furthermore, an FPGA prototype is deployed on the AMD Versal™ VPK180 FPGA to benchmark model-wise performance on real-world DNN workloads (§ IV-C). The evaluation system is also synthesized using the Synopsys Design Compiler® with GlobalFoundries 22FDX® technology, operating at 1GHz and 0.8V with strict timing closure. Post-synthesis simulation is performed for power consumption analysis using Siemens QuestaSim™ and Synopsys PrimeTime® (§ IV-D).

B. Ablation Study for DataMaestro Evaluation

1) *Workload and Evaluation Architecture Setting*: We conduct an ablation study to evaluate the effectiveness of each innovation in *DataMaestro* using 260 different synthetic DNN workloads, categorized into three groups: 1) GeMM, 2) transposed GeMM, and 3) convolution. The synthetic benchmark set features various matrix sizes for GeMM and transposed GeMM, along with diverse feature map sizes, channels, kernel sizes, and strides for convolution, effectively representing typical Transformer and CNN layers. Initially, we disable

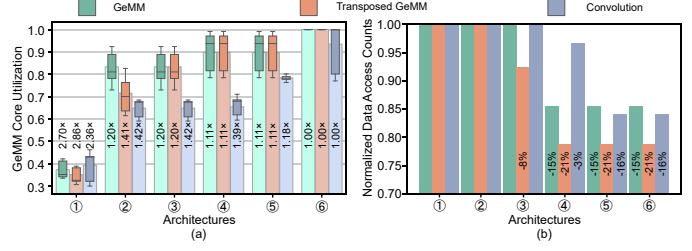


Fig. 7: Ablation study results of GeMM core's utilization distribution presented in box plot, with average utilization annotated in bar chart (a), and normalized data access counts (b) across architectures with different *DataMaestro* features under the synthetic DNN workload set. ① = Baseline (all features turned off); ② = ① + Fine-grained prefetch; ③ = ② + Transposer; ④ = ③ + Broadcaster; ⑤ = ④ + Implicit *im2col*; ⑥ = ⑤ + Addressing mode switching.

all features of *DataMaestros*, architecturally similar to plain data movement units, inside the accelerator system as the baseline (①). Subsequently, we gradually introduce each feature, eventually forming an accelerator system with fully-featured *DataMaestros* (② to ⑥).

2) *Result Analysis*: Figure 7 (a) presents the GeMM core utilization distribution for each DNN kernel group using a box plot, with the average utilization inside each group depicted in the accompanying bar chart. As demonstrated in the results, ② achieves a significant increase in GeMM core utilization, ranging from 1.65× to 2.21× higher than the baseline ① across all workloads, thanks to fine-grained asynchronous prefetch. Among all features, the Transposer is particularly effective for the transposed GeMM workload (③ compared to ②), improving utilization by 1.16×, while Broadcaster increases utilization of all workloads by up to 1.09× (④ compared to ③). The implicit *im2col* [30] transformation further delivers a 1.19× utilization improvement for convolution workloads (⑤ compared to ④). Finally, with addressing mode switching (⑥), the utilization for the two GeMM workloads reaches 100%, with minimal variation across all matrix sizes. For convolution workloads, the average utilization reaches 92.03% but the utilization distribution exhibits notable variation across different workload sizes. Upon further analysis, this fluctuation is attributed to strided convolution layers, which require fetching noncontiguous input data across wider address ranges, resulting in unavoidable bank conflicts. Fortunately, strided convolution layers, typically used for feature map downsampling, comprise only a small portion of real-world DNN workloads and thus have a limited impact, as demonstrated in § IV-C2. Meanwhile, datapath extensions for on-the-fly data manipulation significantly reduce redundant memory accesses, mitigating bank conflicts and saving energy. As illustrated in Figure 7 (b), Transposer reduces the data access counts by 15.86% for transposed GeMM workloads (③ compared to ②), and Broadcaster effectively reduces memory accesses by up to 14.58% across all three workloads (④ compared to ③). Overall, the fully-featured *DataMaestro* (⑥) achieves up to 2.89× performance speedup and up to 21.15% reduction in memory accesses across all test cases, compared to the baseline (①). These results highlight that *DataMaestro* enables versatile and highly efficient data streaming to the accelerator across a variety of DNN kernels with different sizes.

C. Real-world Neural Network Evaluation on FPGA

1) *Evaluation Setup*: We integrate our *DataMaestro* evaluation system into a customized SoC platform derived from [32] and prototype it on the AMD Versal™ VPK180 FPGA, which facilitates rapid and convenient real-world DNN evaluations. The annotated layout and resource utilization of our system are presented in Figure 8.

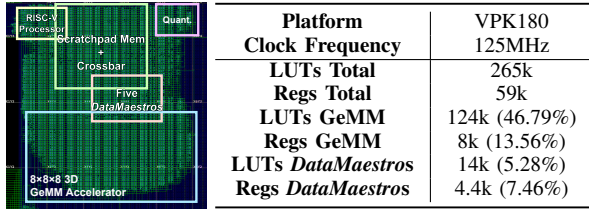


Fig. 8: The FPGA implementation (left) and resource utilization (right) of the *DataMaestro* evaluation system.

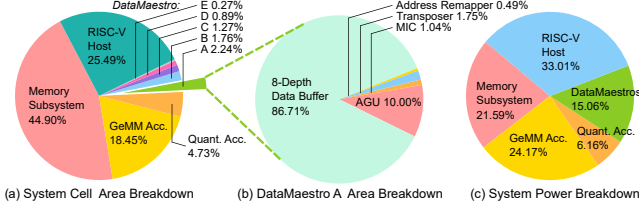


Fig. 9: Breakdown of cell area (a) and total power (c) for the evaluation system, and area composition of *DataMaestro A* (b).

2) *DNN Performance Benchmarking*: We benchmark ResNet-18 [33], VGG-16 [34], ViT-Base-16 [35], and BERT-Base [36] on our FPGA SoC system, with GeMM core utilization listed in Table III. All four networks achieve utilization above 95%, with VGG-16 and ViT-Base-16 reaching nearly 100%. This high GeMM core utilization across these real-world DNN workloads highlights the efficacy of *DataMaestro* in minimizing data stream interruptions, thereby improving the performance of the accelerator system.

TABLE III: GeMM core utilization (in %) of *DataMaestro*-boosted accelerator under real-world DNN workloads on VPK180 FPGA.

	ResNet-18	VGG-16	ViT-B-16	BERT-Base
Type	CNN	CNN	Transformer	Transformer
Utilization (%) [†]	95.45	100.00	99.98	97.85

[[†]] Utilization is calculated as the ratio of theoretical computation cycles without memory stalls to the active cycles of *DataMaestro*.

D. Area and Power breakdown

When synthesized using 22nm FDX technology, our system occupies 0.61mm^2 cell area and consumes 329.4mW of total power when executing an $M=N=K=64$ GeMM (GeMM-64) workload at 1 GHz, achieving a total system-level energy efficiency of 2.57TOPS/W . The detailed system area and power breakdowns are shown in Figure 9 (a) and (c), highlighting the five main components: the GeMM accelerator, Quantization accelerator, five *DataMaestros*, the scratchpad memory subsystem, and the RISC-V host. The first four components form the core of the accelerator system, accounting for 74.52% of the total area and consuming 66.99% of the total power. The five *DataMaestros* collectively occupy only 6.43% of the system area and 15.06% of the total power, with individual area usage ranging from 0.28% to 2.33%. The variation in area consumption demonstrates that *DataMaestro*'s design-time parameters alter the deployed hardware to meet the specific requirements of each data stream, showcasing its high versatility. Furthermore, to investigate the overhead of each feature inside *DataMaestro*, we conduct a detailed analysis of the area composition of a single *DataMaestro A*—the most advanced one among the five instantiations, as shown in Figure 9 (b). The majority of the area is occupied by data FIFOs (87.76%) and MICs (1.04%) to support decoupled fine-grained prefetch. The AGU, responsible for producing the 6-*D* temporal and 2-*D* spatial addresses, accounts for 10% of the total area. The Transposer takes up 1.75%

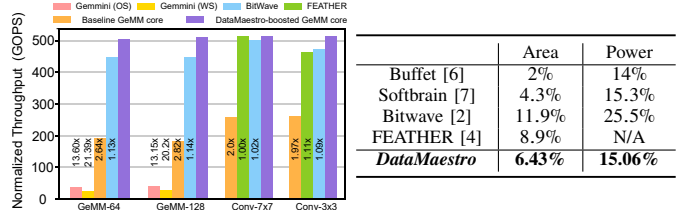


Fig. 10: Comparison of normalized throughput (left) and data movement area and power cost (%) inside the whole accelerator system (right) between SotA solutions and *DataMaestro*. Some data is not revealed in the literature; therefore, the corresponding bars are omitted in (a).

of the total area. Lastly, the address remapper, which is essentially a multiplexer of permuted address bits, occupies a negligible 0.49% of the area.

E. State of the Art Comparison

1) *Comparison with SotA DNN Dataflow Accelerators*: We compare the throughput of our *DataMaestro*-boosted accelerator with SotA DNN dataflow accelerators, including Gemmini in output-stationary (OS) and weight-stationary (WS) modes [1], [37], FEATHER [4], and BitWave [2] using representative DNN kernels. For all systems, throughput is normalized to the same number of PEs (512) and clock frequency (1 GHz) of the accelerator for a fair comparison. As the results in Figure 10 (left) show, our *DataMaestro*-boosted accelerator achieves throughput gains ranging from 1.05 $\times$ to 21.39 $\times$ compared to SotA accelerators across diverse DNN kernels. This significant performance improvement results from the combined efforts of *DataMaestro*'s design-time and runtime flexibility, fine-grained prefetch, and addressing mode switching techniques.

2) *Comparison with SotA Data Streaming Engines*: We further compare data streaming engines in Buffer [6], Softbrain [7] and data movement units in BitWave [2], and FEATHER [4] with *DataMaestro* and summarize their area and power overhead results in Figure 10 (right). The five *DataMaestros* in our evaluation system jointly occupy 5% and 15.58% of the system area and power consumption, which is competitive with SotA solutions, while flexibly and efficiently streamlining data to boost the dataflow accelerator's performance.

V. CONCLUSION

In this paper, we present *DataMaestro*, a versatile, efficient, and open-source data streaming unit that brings the decoupled access/execute architecture to DNN dataflow accelerators. *DataMaestro* supports a programmable *N*-Dimensional access pattern to accommodate diverse workload types and dataflows, incorporates fine-grained prefetch and addressing mode switching to mitigate bank conflicts, and enables customizable on-the-fly data manipulation to reduce memory footprints and accesses. We integrate five *DataMaestros* with a Tensor Core-like GeMM accelerator and a Quantization accelerator into a RISC-V host system. FPGA prototyping and VLSI synthesis results demonstrate that *DataMaestro* helps accelerators achieve nearly 100% utilization, which is 1.05 – 21.39 $\times$ higher than SotA solutions, with area and energy cost of merely 6.43% and 15.06% of the total system.

ACKNOWLEDGMENT

This project has been partly funded by the European Research Council (ERC) under grant agreement No. 101088865, the European Union's Horizon 2020 program (CONVOLVE) under grant agreement No. 101070374, the Flanders AI Research Program, and KU Leuven.

REFERENCES

- [1] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao *et al.*, “Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 769–774.
- [2] M. Shi, V. Jain, A. Joseph, M. Meijer, and M. Verhelst, “Bitwave: Exploiting column-based bit-level sparsity for deep learning acceleration,” in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 732–746.
- [3] F. Schneider, M. Karagounis, and B. Choubey, “Energy and bandwidth efficient sparse programmable dataflow accelerator,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.
- [4] J. Tong, A. Itagi, P. Chatarasi, and T. Krishna, “Feather: A reconfigurable accelerator with data reordering support for low-cost on-chip dataflow switching,” in *Proceedings of the 51th Annual International Symposium on Computer Architecture*, ser. ISCA ’24. Argentina: Association for Computing Machinery, 2024.
- [5] F. Schuiki, F. Zaruba, T. Hoefler, and L. Benini, “Stream semantic registers: A lightweight risc-v isa extension achieving full compute utilization in single-issue cores,” *IEEE Transactions on Computers*, vol. 70, no. 2, pp. 212–227, 2020.
- [6] M. Pellauer, Y. S. Shao, J. Clemons, N. Crago, K. Hegde, R. Venkatesan, S. W. Keckler, C. W. Fletcher, and J. Emer, “Buffets: An efficient and composable storage idiom for explicit decoupled data orchestration,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 137–151.
- [7] T. Nowatzki, V. Gangadhar, N. Ardalani, and K. Sankaralingam, “Stream-dataflow acceleration,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017, pp. 416–429.
- [8] P. Houshmand, G. M. Sarda, V. Jain, K. Ueyoshi, I. A. Papistas, M. Shi, Q. Zheng, D. Bhattacharjee, A. Mallik, P. Debacker, D. Verkest, and M. Verhelst, “Diana: An end-to-end hybrid digital and analog neural network soc for the edge,” *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 203–215, 2023.
- [9] T. Norrie, N. Patil, D. H. Yoon, G. Kurian, S. Li, J. Laudon, C. Young, N. P. Jouppi, and D. A. Patterson, “Google’s training chips revealed: Tpuv2 and tpuv3,” in *Hot Chips Symposium*, 2020, pp. 1–70.
- [10] C.-Y. Du, C.-F. Tsai, W.-C. Chen, L.-Y. Lin, N.-S. Chang, C.-P. Lin, C.-S. Chen, and C.-H. Yang, “A 28nm 11.2 tops/w hardware-utilization-aware neural-network accelerator with dynamic dataflow,” in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2023, pp. 1–3.
- [11] E. Talpes, D. D. Sarma, G. Venkataramanan, P. Bannon, B. McGee, B. Floering, A. Jalote, C. Hsiong, S. Arora, A. Gorti *et al.*, “Compute solution for tesla’s full self-driving computer,” *IEEE Micro*, vol. 40, no. 2, pp. 25–35, 2020.
- [12] H. Kwon, A. Samajdar, and T. Krishna, “Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects,” *ACM SIGPLAN Notices*, vol. 53, no. 2, pp. 461–475, 2018.
- [13] Q. Huang, P.-A. Tsai, J. S. Emer, and A. Parashar, “Mind the gap: Attainable data movement and operational intensity bounds for tensor algorithms,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 150–166.
- [14] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, “Timeloop: A systematic approach to dnn accelerator evaluation,” in *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 2019, pp. 304–315.
- [15] L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst, “Zigzag: Enlarging joint architecture-mapping design space exploration for dnn accelerators,” *IEEE Transactions on Computers*, vol. 70, no. 8, pp. 1160–1174, 2021.
- [16] R. Hameed, W. Qadeer, M. Wachs, O. Azizi, A. Solomatnikov, B. C. Lee, S. Richardson, C. Kozyrakis, and M. Horowitz, “Understanding sources of inefficiency in general-purpose chips,” in *Proceedings of the 37th annual international symposium on Computer architecture*, 2010, pp. 37–47.
- [17] Y. Tortorella, L. Bertaccini, L. Benini, D. Rossi, and F. Conti, “Redmule: A mixed-precision matrix-matrix operation engine for flexible and energy-efficient on-chip linear algebra and tinyml training acceleration,” *Future Generation Computer Systems*, vol. 149, pp. 122–135, 2023.
- [18] H. Liao, J. Tu, J. Xia, H. Liu, X. Zhou, H. Yuan, and Y. Hu, “Ascend: a scalable and unified architecture for ubiquitous deep neural network computing: Industry track paper,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 789–801.
- [19] J. E. Smith, “Decoupled access/execute computer architectures,” *ACM SIGARCH Computer Architecture News*, vol. 10, no. 3, pp. 112–119, 1982.
- [20] J. M. Domingos, N. Neves, N. Roma, and P. Tomás, “Unlimited vector extension with data streaming support,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 209–222.
- [21] Z. Wang and T. Nowatzki, “Stream-based memory access specialization for general purpose processors,” in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 736–749.
- [22] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” 2017. [Online]. Available: <https://arxiv.org/abs/1703.09039>
- [23] N. Corporation, “Nvidia turing architecture whitepaper,” <https://images.nvidia.com/>, 2018.
- [24] H. Ye, X. Zhang, Z. Huang, G. Chen, and D. Chen, “Hybridnn: A framework for high-performance hybrid dnn accelerator design and implementation,” in *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2020, pp. 1–6.
- [25] M. Shi, S. Coleman, C. VanDeMierop, A. Joseph, M. Meijer, W. Dehaene, and M. Verhelst, “Cmds: Cross-layer dataflow optimization for dnn accelerators exploiting multi-bank memories,” in *2023 24th International Symposium on Quality Electronic Design (ISQED)*. IEEE, 2023, pp. 1–8.
- [26] M. D. Lam, E. E. Rothberg, and M. E. Wolf, “The cache performance and optimizations of blocked algorithms,” *ACM SIGOPS Operating Systems Review*, vol. 25, no. Special Issue, pp. 63–74, 1991.
- [27] H. Zheng, S. Oh, H. Wang, P. Briggs, J. Gai, A. Jain, Y. Liu, R. Heaton, R. Huang, and Y. Wang, “Optimizing memory-access patterns for deep learning accelerators,” *arXiv preprint arXiv:2002.12798*, 2020.
- [28] Z. Zhang, Z. Zhu, and X. Zhang, “A permutation-based page interleaving scheme to reduce row-buffer conflicts and exploit data locality,” in *Proceedings of the 33rd annual ACM/IEEE international symposium on Microarchitecture*, 2000, pp. 32–41.
- [29] F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini, “Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads,” *IEEE Transactions on Computers*, vol. 70, no. 11, pp. 1845–1860, 2020.
- [30] Y. Zhou, M. Yang, C. Guo, J. Leng, Y. Liang, Q. Chen, M. Guo, and Y. Zhu, “Characterizing and demystifying the implicit convolution algorithm on commercial matrix-multiplication accelerators,” in *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2021, pp. 214–225.
- [31] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzyniak, and K. Asanović, “Chisel: constructing hardware in a scala embedded language,” in *Proceedings of the 49th Annual Design Automation Conference*, 2012, pp. 1216–1225.
- [32] G. Paulin, P. Scheffler, T. Benz, M. Cavalcante, T. Fischer, M. Eggmann, Y. Zhang, N. Wistoff, L. Bertaccini, L. Colagrande *et al.*, “Occamy: A 432-core 28.1 dp-gflop/s/w 83% fpu utilization dual-chiplet, dual-hbm2e risc-v-based accelerator for stencil and sparse linear algebra computations with 8-to-64-bit floating-point support in 12nm finfet,” in *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 2024, pp. 1–2.
- [33] K. He, X. Zhang, S. Ren, and J. Sun, “Identity mappings in deep residual networks,” in *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14*. Springer, 2016, pp. 630–645.
- [34] K. Simonyan, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [35] A. Dosovitskiy, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020.
- [36] J. Devlin, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [37] A. Gonzalez, J. Zhao, B. Korpan, H. Genc, C. Schmidt, J. Wright, A. Biswas, A. Amid, F. Sheikh, A. Sorokin *et al.*, “A 16mm 2 106.1 gops/w heterogeneous risc-v multi-core multi-accelerator soc in low-power 22nm finfet,” in *ESSCIRC 2021-IEEE 47th European Solid State Circuits Conference (ESSCIRC)*. IEEE, 2021, pp. 259–262.